Software Impacts 28 (2026) 100844

Contents lists available at ScienceDirect

Software Impacts

journal homepage: www.journals.elsevier.com/software-impacts

Original Software Publication

InsuranceRAG: A multi-module retrieval-augmented AI system for intelligent insurance assistance

Sejal Hanmante, Shivani Patil, Priyanka V. Deshmukh, Aniket K. Shahade *

Symbiosis Institute of Technology, Pune Campus, Symbiosis International (Deemed University), Pune, India

ARTICLE INFO

Keywords:
Retrieval-augmented generation
Health insurance software
Natural language processing
Policy recommendation system
Document retrieval
Open-source software

ABSTRACT

This article presents InsuranceRAG, an open-source software system designed to support intelligent interaction with health insurance data using Retrieval-Augmented Generation (RAG). The software provides a Flask-based web interface that enables users to submit natural language queries, upload insurance policy documents, and receive context-aware responses. It follows a modular architecture comprising independent components for conversational assistance, policy recommendation, and document retrieval, all coordinated through a centralized backend. Shared RAG services integrate text embedding models, FAISS-based semantic search, and configurable large language model inference. The open-source implementation promotes reproducibility, extensibility, and practical adoption in insurance-focused natural language processing applications.

Code metadata

Current code version	*V1.0*
Permanent link to code/repository used for this code version	https://github.com/SoftwareImpacts/SIMPAC-2026-9
Permanent link to reproducible capsule	
Legal code license	*MIT License*
Code versioning system used	None
Software code languages, tools and services used	Python, Flask, FAISS, Sentence Transformers, SQLite, HTML, CSS, OpenRouter API
Compilation requirements, operating environments and dependencies	**Python** ≥ 3.9; Flask, sentence-transformers, faiss-cpu/faiss-gpu, numpy, pandas, scikit-learn, sqlite3, PyPDF2/pdfplumber, requests
If available, link to developer documentation/manual	https://github.com/SEJALHANMANTE/NLPINSURANCE-FINTECHPROJ#readme
Support email for questions	sejalhanmante@gmail.com

1. Introduction

The rapid digitalization of the insurance and financial technology (FinTech) sectors has intensified the need for intelligent systems capable of assisting users in understanding complex policy documents, selecting suitable insurance products, and resolving domain-specific queries through natural language interaction. Health insurance policies, in particular, are often characterized by lengthy documents, technical terminology, and intricate clauses that are difficult for non-expert users to interpret correctly, frequently leading to confusion and suboptimal decision-making [1,2]. As a result, there has been growing interest in applying Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques to improve transparency, accessibility, and user engagement in insurance services [3,4].

Recent advances in Large Language Models (LLMs) have significantly enhanced conversational AI systems, enabling more fluent and context-aware responses in customer-facing applications [5]. However, purely generative models are prone to hallucinations and factual inconsistencies, particularly in knowledge-intensive domains such as insurance, where accuracy and traceability are critical [6,7]. Retrieval-Augmented Generation (RAG) has emerged as an effective solution to mitigate these limitations by grounding model outputs in externally retrieved, domain-specific information [8,9]. RAG-based approaches have demonstrated improved relevance and factual consistency in tasks such as document interpretation, policy recommendation, and question answering across specialized domains [10–12].

* Corresponding author.
E-mail addresses: sejalhanmante@gmail.com (S. Hanmante), shivanipatil309@gmail.com (S. Patil), priyanka.deshmukh@sitpune.edu.in (P.V. Deshmukh), aniket.shahade@sitpune.edu.in (A.K. Shahade).

https://doi.org/10.1016/j.simpa.2026.100844
Received 14 February 2026; Received in revised form 5 May 2026; Accepted 6 May 2026

2665-9638/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

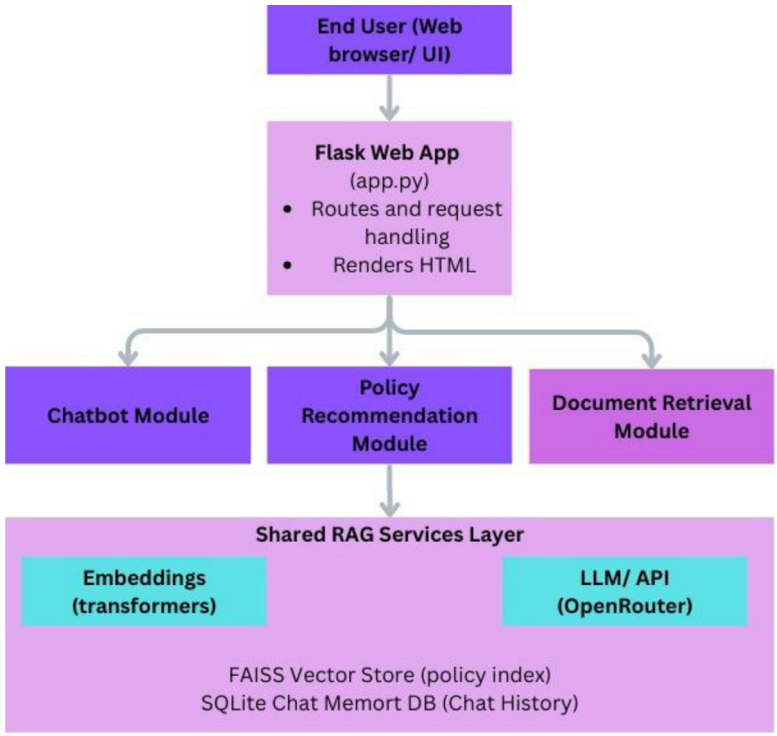


Fig. 1. Workflow pipeline.

Despite these advancements, most existing contributions in literature primarily focus on methodological improvements and evaluation metrics, with limited emphasis on delivering reusable, end-to-end software systems that can be deployed, extended, and studied by the broader research and practitioner community [13]. In the context of health insurance, publicly available tools that integrate conversational assistance, personalized policy recommendation, and clause-level document retrieval within a single, modular software framework remain scarce. Furthermore, many implementations lack transparent architectural documentation, making reproducibility and practical adoption challenging.

To address these gaps, this article presents InsuranceRAG, an open-source software system designed to support intelligent interaction with health insurance data using Retrieval-Augmented Generation.

By releasing the complete implementation as an open-source software artifact, this work aims to facilitate reproducible research, accelerate prototyping of insurance-focused NLP applications, and provide a practical foundation for future developments in InsurTech and FinTech systems.

1.1. Software description

The system provides a Flask-based web interface through which users can submit natural language queries, upload insurance policy documents, and receive context-aware responses. Internally, the software follows modular architecture comprising independent components for conversational assistance, policy recommendation, and policy document retrieval, all orchestrated by a centralized backend controller. These components share common RAG services, including text embedding models, vector-based semantic search, and large language model inference, enabling consistent knowledge grounding and extensibility.

Fig. 1 presents the high-level system architecture of InsuranceRAG, highlighting the interaction between the web interface, core modules, and shared retrieval-augmented generation components.

2. Overall architecture

The software follows a layered architecture comprising a user interface layer, a backend application controller, independent functional modules, shared RAG services, and persistent storage. Users interact with the system through a Flask-based web interface that supports text-based queries and insurance policy document uploads. Incoming requests are handled by a centralized application controller, which routes each request to the appropriate functional module based on the selected task. Each functional module operates independently while sharing common RAG services, including text embedding models, vector-based semantic search, and large language model inference. This design ensures consistency across modules while allowing individual components to be modified or extended without affecting the rest of the system.

3. User interface and backend controller

The user interface is implemented using Flask with HTML and CSS templates, providing a simple web-based front end for interacting with the system. Users can submit natural language queries, select the desired operation (chat assistance, policy recommendation, or document retrieval), and upload insurance policy documents in PDF format.

The backend controller, implemented in the main application script, manages request handling, input validation, and orchestration of module execution. It acts as an intermediary between the user interface and the underlying functional modules, ensuring modular separation between presentation logic and core processing.

4. Functional modules

InsuranceRAG is organized into three primary functional modules, each designed to address a distinct user interaction scenario within the health insurance domain. The modules are implemented as independent

Python components and are invoked by the backend application controller based on the user's selected task. While each module performs a specific function, all modules rely on a shared Retrieval-Augmented Generation (RAG) service layer for embedding generation, semantic retrieval, and response generation. This modular design improves code maintainability, reuse, and extensibility.

5. Conversational insurance assistant module

The conversational insurance assistant module enables users to ask general health insurance-related questions using natural language. Typical queries include explanations of insurance terms, coverage-related clarifications, and high-level policy information. Upon receiving a user query, the module first preprocesses the input text and converts it into a dense vector representation using a pre-trained sentence embedding model.

The generated embedding is compared against previously stored question embeddings using cosine similarity to identify semantically related queries. If relevant historical context is available, it is retrieved to support conversational continuity. The retrieved context, together with the original user query, is then passed to a large language model through the RAG pipeline to generate a grounded, context-aware response. The module optionally stores the query–response pair in a lightweight SQLite database, enabling efficient reuse of prior interactions and reducing redundant inference for repeated or similar queries.

This module is designed to be model-agnostic, allowing developers to replace the embedding model or language model interface with minimal changes to the codebase.

6. Policy recommendation module

The policy recommendation module is responsible for suggesting suitable health insurance policies based on user-specified requirements, such as coverage amount, budget range, number of insured individuals, or specific benefits. The module operates on structured and semi-structured policy data, including policy descriptions and associated metadata.

During initialization, policy descriptions are transformed into dense vector embeddings and indexed using FAISS to enable efficient semantic similarity search. When a user submits a recommendation query, the query is embedded and compared against the indexed policy vectors to retrieve the most relevant policy candidates. These retrieved policies, along with their key attributes, are incorporated into the RAG pipeline to generate explanatory recommendations that justify why certain policies are suitable for the user's needs.

The recommendation module is designed to support easy integration of additional policy datasets and alternative retrieval strategies. New policies can be added by updating the data source and regenerating the vector index, without requiring changes to the core application logic.

7. Document retrieval module

The document retrieval module enables clause-level information extraction from uploaded health insurance policy documents. Users can upload policy documents in PDF format through the web interface, after which the module performs text extraction using document parsing utilities. The extracted text is segmented into smaller, semantically coherent chunks to improve retrieval accuracy.

Each text segment is embedded and stored in a vector index, allowing efficient semantic matching against user queries. When a query is submitted, the module retrieves the most relevant document segments based on embedding similarity and forwards them to the RAG pipeline. The language model then generates a concise, context-aware response grounded in the retrieved policy clauses, enabling users to obtain targeted explanations without manually navigating lengthy documents.

This module is particularly useful for interpreting complex policy terms and conditions and can be extended to support additional document formats or storage backends.

8. 2Module interaction and orchestration

All functional modules are orchestrated through a centralized backend controller, which manages request routing, data flow, and interaction with shared RAG services. By decoupling the functional modules from the user interface and shared services, the software ensures that individual components can be independently modified, tested, or replaced. This architectural separation supports extensibility and makes the system suitable for experimentation, educational use, and early-stage InsurTech prototyping.

The key configuration parameters governing embedding generation, semantic retrieval, memory management, and LLM-based response generation are summarized in Table 1.

The system employs different large language model backends across its three functional modules, selected based on availability and task requirements. The conversational chatbot module uses DeepSeek-R1 (free tier) accessed via the OpenRouter API, with a sampling temperature of 0.3 to produce focused and consistent responses. The policy recommendation module uses LLaMA 3 70B (llama3-70b-8192) served through the Groq API, configured with a temperature of 0.4 and a maximum output length of 512 tokens. The document retrieval module uses LLaMA 4 Scout (free tier) via OpenRouter for clause-level summarization. All models are invoked through their respective REST APIs and can be substituted with alternative backends by modifying the configuration parameters.

The chatbot module was evaluated on 50 queries sampled from the internal insurance QA dataset (decoded_questions.json), which contains curated question–answer pairs covering Indian health insurance terminology, coverage, and policy clauses. The policy recommendation module was evaluated on 20 user requirement queries against the synthetic policy dataset indexed in faiss_policy.index (derived from the Policy Recommendation Dataset folder). The document retrieval module was evaluated on 30 queries submitted against uploaded policy PDFs, which are chunked into 3-sentence segments prior to indexing. **Hit@5** is defined as 1 if at least one annotated relevant item appears in the top-5 retrieved results, averaged across all queries. **Recall@5** is the fraction of all relevant annotated items recovered within the top-5 results. Evaluator agent scores (Table 3) were produced using a separate LLM instance rating each response on a 1–5 scale across four qualitative dimensions.

Table 2 reports the retrieval and response performance of the three core modules, including Hit@5, Recall@5, semantic similarity (BERTScore), and average response latency. The Hit@5 score of 1.00 for the document retrieval and policy recommendation modules reflects the constrained nature of their respective test sets. In both cases, evaluation queries were formulated against the same corpus used to build the FAISS index — the uploaded policy PDF chunks for document retrieval, and the synthetic policy_metadata.json for policy recommendation — making it highly likely that relevant content appears within the top-5 results. These scores should therefore be interpreted in the context of closed-corpus retrieval rather than as claims of generalized performance. Future evaluations will use independently constructed query sets with out-of-distribution documents.

As shown in Table 3, all modules achieve consistently high qualitative scores, indicating clear, accurate, and helpful responses.

8.1. Impact overview

InsuranceRAG delivers measurable impact as the complete open-source implementation of the system architecture originally proposed and validated in our prior work [14]. In that study, the software served as the experimental platform for demonstrating that multi-module Retrieval-Augmented Generation can effectively address factual consistency challenges in health insurance applications, achieving high retrieval accuracy (Hit@5 scores of 0.92–1.00) and strong response

Table 1
Default system configuration parameters.

Parameter	Description	Value
Number of modules	Chatbot, Document Retrieval, Policy Recommendation	3
Embedding model	SentenceTransformer/ InsuranceBERT	all-MiniLM-L6-v2/ industry-bert
Embedding dimension	Size of embedding vectors	384/ 768
Similarity metric	Retrieval similarity	Cosine similarity
FAISS index type	Vector index	IndexFlatIP
Memory backend	Conversational memory	SQLite
Cache size	Stored chatbot interactions	Dynamic
Top-k retrieval	Number of retrieved documents	3–5
LLM backend	Response generation	DeepSeek/ LLaMA

Table 2
Retrieval and recommendation performance.

Module	Queries	Hit@5	Recall@5	BERTScore (F1)	Avg Latency (ms)
Chatbot retrieval	50	0.92	0.88	0.84	420
Document retrieval	30	1.00	0.83	0.84	510
Policy recommendation	20	1.00	0.83	0.84	460

Table 3
Evaluator agent scores.

Module	Relevance	Accuracy	Clarity	Helpfulness
Chatbot	4.6	4.5	4.7	4.6
Doc Retrieval	4.7	4.6	4.8	4.7
Policy Recommendation	4.5	4.4	4.6	4.5

quality (BERTScore F1 of 0.84) across all functional modules. By releasing this tested implementation under the MIT License, the work enables researchers to reproduce our published findings, conduct comparative studies with alternative embedding models or retrieval strategies, and extend the framework to adjacent domains. For educators, the modular architecture provides a hands-on resource for teaching end-to-end AI system development, while industry practitioners can leverage the software as a proven foundation for prototyping transparent, explainable insurance assistance tools. The open-source release thus supports the broader goals of reproducible research and accelerated innovation at the intersection of NLP and insurance technology.

9. Limitations and future development

While InsuranceRAG provides a practical and extensible software framework for interacting with health insurance data, several limitations should be acknowledged. The quality of system responses depends on the completeness and accuracy of the underlying policy data and uploaded documents. As with most Retrieval-Augmented Generation systems, the software may still produce suboptimal outputs if relevant information is missing or poorly structured. In addition, the current implementation primarily supports English-language content and assumes text-based policy documents in PDF format.

The current implementation has known scalability constraints tied to its storage and indexing design. The conversational memory module uses SQLite (chatbot_memory.db), which does not support concurrent multi-user access and is unsuitable for high-throughput deployments. The document retrieval module constructs a FAISS IndexFlatIP index entirely in-memory at runtime; this index is not persisted across sessions, requiring re-embedding and re-indexing of uploaded documents on every run, and its linear search complexity will degrade with larger document collections. The policy recommendation module loads a pre-built FAISS index from disk (faiss_policy.index), which improves persistence but still resides fully in RAM during execution. For production-scale use, migrating to a persistent vector store and a scalable database backend would be recommended.

Insurance policy documents uploaded by users are stored in the local uploads/ directory on the server filesystem during processing. The current implementation does not implement encryption of stored files, per-session user isolation, or automatic deletion of uploaded documents after processing. While this is acceptable for local research and prototyping use, any broader deployment involving real user documents — which may contain policy numbers, names, or other personally identifiable information — would require transport-layer encryption (HTTPS), strict access controls, post-processing file deletion, and compliance with applicable data protection frameworks such as GDPR or India's Digital Personal Data Protection Act (DPDPA 2023). These security enhancements are planned as part of future development.

Future development will focus on extending multilingual support, improving document preprocessing for diverse policy formats, and integrating additional large language models, including locally hosted alternatives. Enhancements to storage scalability and support for real-world policy datasets are also planned to facilitate deployment in more complex production environments.

10. Conclusion

This software introduces a unified, retrieval-augmented AI framework that seamlessly integrates chatbot assistance, personalized policy recommendation, and clause-level document retrieval for the insurance domain. By combining domain-adapted embeddings, FAISS-based retrieval, large language models, and an evaluator agent, the system delivers accurate, explainable, and user-focused insurance support. The architecture establishes a scalable blueprint for deploying trustworthy AI systems in regulated, document-heavy domains.

11. Relationship to prior work

The software described in this article is based on the system architecture and methodology previously presented in our research publication [14], while the present work focuses exclusively on the released software artifact, its implementation, and practical usage. No experimental results or evaluations are duplicated, and the two articles serve complementary purposes: one as a scientific study and the other as a software publication.

CRediT authorship contribution statement

Sejal Hanmante: Writing – original draft, Visualization, Software, Project administration, Methodology, Investigation, Formal analysis, Conceptualization. **Shivani Patil:** Writing – review & editing, Validation, Resources, Methodology, Investigation, Conceptualization. **Priyanka V. Deshmukh:** Writing – review & editing, Conceptualization. **Aniket K. Shahade:** Writing – review & editing, Supervision, Resources, Project administration, Formal analysis.

Funding

No Funding

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Express Healthcare, The future of health insurance: Integrating AI for fair pricing and better coverag, Express Healthcare.
[2] A. Poddar, The State of Insurance and Healthcare in India, PlumHQ.
[3] S. Kolambe, P. Kaur, Insurance policy summarization using natural language processing (NLP), Front. Health Inform. 13 (3) (2024) [Online]. Available: www.healthinformaticsjournal.com.
[4] M. Feng, B. Xiang, M.R. Glass, L. Wang, B. Zhou, Applying deep learning to answer selection: A study and an open task, 2015.
[5] C. Cao, et al., LLMs for Insurance: Opportunities, Challenges and Concerns.
[6] Z. Zhang, et al., Personalization of large language models: A survey, 2025.
[7] S. Prabhune, D.J. Berndt, Deploying large language models with retrieval augmented generation, 2024.
[8] H. Frank, K. Ämnesgranskare, D.V. D'aurelio, E. Andrésdóttir, E. Furberg, J. Hedberg, Teknisk-naturvetenskapliga fakulteten Uppsala universitet, Utgivningsort Uppsala/Visby Automated extraction of insurance policy information.
[9] M. Nuruzzaman, O.K. Hussain, IntelliBot: A dialogue-based chatbot for the insurance industry, Knowl Based Syst 196 (2020) 105810.
[10] W. Zhang, J. Shi, X. Wang, H. Wynn, AI-powered decision-making in facilitating insurance claim dispute resolution, Ann. Oper. Res. (2023) http://dx.doi.org/10.1007/s10479-023-05631-9.
[11] B. Chandra, P. Preethika, S. Challagundla, Y. Gogireddy, End-to-End Neural Embedding Pipeline for Large-Scale PDF Document Retrieval Using Distributed FAISS and Sentence Transformer Models, Journal ID, 1004, p. 1429.
[12] S. Bedi, et al., A systematic review of testing and evaluation of healthcare applications of large language models (LLMs), 2024, http://dx.doi.org/10.1101/2024.04.15.24305869.
[13] R.P. Pramila, R.S.R., V.S., Artificial intelligence and natural language processing (NLP) integrated multilingual health insurance application, in: 2024 3rd International Conference on Automation, Computing and Renewable Systems, ICACRS, IEEE, 2024, pp. 1141–1145, http://dx.doi.org/10.1109/ICACRS62842.2024.10841786.
[14] S. Hanmante, S. Patil, A.K. Shahade, A multi module a.i. system for intelligent health insurance support using retrieval augmented generation, Sci Rep (2025) http://dx.doi.org/10.1038/s41598-025-31038-6.